

Modul SYNESIS

Bedah konsep machine learning

Sandy Fauzi Amrulloh

Agustus 2026

Contents

Modul Inti: Bedah Konsep Machine Learning (Bahasa Manusia)	1
Bagian 0. Satu Analogi buat Segalanya	1
Bagian 1. Basic Mesin Pintar	2
Bagian 2. Paham vs Cuma Hafalan (Overfitting)	3
Bagian 3. Masuk ke Jaringan Saraf (Neural Network)	3
Bagian 4. NLP, Klasifikasi & Jarak	3
Bagian 5 & 6. Suara dan Wajah	4
Bagian 7. LLM & Mesin Pemaham Bahasa	4
Bagian 8. JARVIS (Agent System)	5
Bagian 9. Kamus Fisika ke Machine Learning	5
Bagian 10. Tes Kejujuran (Apakah Lu Beneran Paham?)	6

Modul Inti: Bedah Konsep Machine Learning (Bahasa Manusia)

Dokumen ini bukan silabus atau daftar tugas, melainkan contekan konsep pencerahan. Kalau Silabus ngasih tahu *apa* yang harus dikerjain, dokumen ini ngasih tahu *gimana cara ngebayanginnya* di otakmu tanpa harus pusing dicekoki rumus duluan.

Cara pakainya gampang: baca satu bagian, tutup layar, terus coba jelasin ulang ke tembok pakai bahasamu sendiri. Kalau kamu di tengah jalan tiba-tiba macet atau *ngelag*, itu tandanya kamu cuma hafal nama istilah elitnya doang, tapi aslinya belum paham bentuk barangnya.

Bagian 0. Satu Analogi buat Segalanya

Sebelum kita kejauhan, tolong pegang satu konsep ini baik-baik: **Bayangin sebuah kelereng yang digelindingin ke dalam mangkok.** Dia bakal meluncur turun, mentul-mentul dikit, lalu pada akhirnya diam berhenti di dasar mangkok. Kenapa? Karena hukum alam mewajibkan kelereng selalu mencari titik energi potensial paling rendah.

Nah, **seluruh jagat raya Machine Learning itu intinya cuma kelereng di dalam mangkok ini.**

Entah itu regresi linear anak kuliah, Face ID di HP-mu, sampai model raksasa kayak ChatGPT. Semuanya murni cuma proses nggelindingin kelereng ke dasar mangkok. Bedanya cuma dua hal:

1. **Dimensinya beda.** Kalau regresi linear, mangkoknya bentuk 2D atau 3D biasa yang gampang digambar. Kalau model LLM miliaran parameter, mangkoknya punya miliaran dimensi yang mustahil dibayangin otak manusia biasa. 2. **Bentuknya berantakan.** Mangkok regresi linear itu mulus. Tapi mangkok Neural Network itu gronjal-gronjal kayak pegunungan yang penuh tebing, jurang, dan “lembah palsu” (lokal minimum). Kelereng kita bisa aja nyangkut di lembah dangkal padahal aslinya ada dasar yang lebih dalam di sebelahnya.

Itu doang intinya. Sisa dari dokumen ini murni cuma ngebahas tiga hal: gimana cara ngebentuk mangkoknya, gimana cara ngerasain kemiringannya di tengah kegelapan, dan gimana ngatur langkah si kelereng biar jatuhnya pas (nggak mental ke luar angkasa).

Bagian 1. Basic Mesin Pintar

Model itu Cuma Mesin Ber-Kenop

Bayangin sebuah kotak misterius yang punya satu corong input, satu corong output, dan banyak kenop puteran (parameter) di atasnya. Masukin angka, keluar angka lain. Angka apa yang keluar? Ya sepenuhnya tergantung ke mana posisi kenopnya kamu puter. Itulah model ML. Regresi cuma punya 2 kenop (w dan b). GPT punya miliaran kenop. “Melatih model” (Training) itu aslinya ya cuma usaha nyari posisi putaran kenop yang paling *perfect* secara otomatis.

Loss itu Papan Skor Error

Biar komputer bisa muter-muter kenopnya secara mandiri, dia harus tahu seberapa “goblok” tebakannya saat ini. Kita butuh skor kesalahan (Loss). Makin kecil angkanya, makin bagus. Contoh populernya adalah **MSE (Mean Squared Error)**: selisih tebakan vs jawaban asli, dikuadratin, terus dirata-rata total. Kenapa wajib dikuadratin? Biar minus sama plus nggak saling menetralsir, dan karena sistem kuadrat bikin denda error gede makin meledak bengkok. Jadi komputer bakal super ketakutan bikin error gede.

Gradien itu Rabaan Kemiringan Tanah

Bayangin kamu lagi terjebak di tebing gunung pas kabut tebal dan buta arah. Kamu cuma bisa ngerasain tanah di bawah tapak sepatumu ini miring ke arah mana. Nah, insting kemiringan inilah yang namanya **Gradien**. Gradien ngasih petunjuk arah “tanjakan yang paling curam”. Karena tujuan kita mau meluncur turun ke dasar lembah, ya kita ambil langkah ke **arah kebalikannya** (makanya di semua rumus ada tanda minus -).

Learning Rate (LR) itu Lebar Langkah Kakimu

Kalau gradien nentuin arahnya, LR itu penentu seberapa lebar kamu ngelangkah. - **Kekecilan:** Kamu jalannya ngingsot semili-semili. Butuh sejuta langkah buat nyampe yang bikin komputermu nangis kelamaan. - **Kebesaran:** Kamu lompatnya kelewat semangat, yang ada malah bablas ngelewatin lembahnya dan nabrak tebing seberang (Loss malah naik/Divergen). Persis kayak ayunan pegas kurang redaman yang goyangannya makin lama makin liar ngerusak sistem.

Bagian 2. Paham vs Cuma Hafalan (Overfitting)

Model yang nilai errornya super kecil (hampir nol) pas dilatih itu malah bahaya. Ibarat ada murid yang dapet nilai 100 pas ujian. Bukan karena dia pinter, tapi karena dia nyolong dan ngapalin *plek-ketiplek* kunci jawaban soal ujian tahun lalu. Begitu besoknya diuji pakai soal tryout yang beda dikit, nilainya jeblok hancur lebur. Penyakit ini di Machine Learning namanya **Overfitting**.

Makanya data kita selalu dibagi dua: Data Latih (buat belajar) dan Data Uji (buat ujian). Kalau grafik nilainya jago di latih tapi nyungsep di uji, fix modelmu cuma mesin menghafal.

Solusinya: Regularisasi (L2). Anggap aja kita ngiket semua puteran kenop modelnya pakai karet gelang/pegas biar posisinya selalu ketarik balik ke angka nol. Jadi, si model nggak bisa lagi sembarangan muter kenop ke angka lebay ekstrem cuma demi nge-pas-pasin tebakan ke titik data acak. Konsep ini literally ekuivalen persis sama Hukum Hooke tentang pegas di Fisika!

Bagian 3. Masuk ke Jaringan Saraf (Neural Network)

Kenapa Perlu Ditekuk (Fungsi Aktivasi)?

Sebanyak apapun penggaris lurus kamu tumpuk, selamanya dia nggak akan bisa ngebingkai bentuk bundar. Makanya kita butuh nyelipin **Fungsi Aktivasi** (contoh: ReLU). Aturannya cupu banget: *kalau nilainya minus paksa jadi 0, kalau plus biarin aja*. Efeknya? Ini ngasih “tekukan” atau patahan yang tajam. Gabungan jutaan patahan inilah yang bikin model bisa membentuk kurva meliuk-liuk serumit apa pun buat misahin data.

Jaringan Saraf Tiruan (Neural Network) sejatinya cuma rutinitas membosankan ini: Kali matriks -> Tekuk -> Kali Matriks -> Tekuk -> Ulangi terus -> Keluarin jawaban. To long stop bayangin sel saraf biologis otak manusia yang berkedip-kedip saling kirim sinyal mistis. Nggak ada keajaiban mistis di sini, semuanya murni perkalian matriks kaku.

Backpropagation & Autograd

Kalau model GPT punya bermiliar-miliar kenop, masa kita harus ngetes muterin kenopnya satu per satu buat ngelihat efeknya ke skor Loss? Keburu Bumi hancur. Solusi cerdasnya: **Jalan Mundur (Backpropagation)**. Mengandalkan *Aturan Rantai* kalkulus, tapi ngitungnya disetir dari belakang (titik Loss) memutar balik ke titik awal. Cukup *sekali jalan mundur*, jatah turunan/efek buat milyaran kenop langsung kejawab serentak.

Nah, biar kita nggak keriting nurunin rumus kalkulus tebal-tebal, pakailah **Autograd**. Ini fitur tukang catet (buku harian) yang nulis silsilah tiap angka saat proses hitung maju, lalu dia sendiri yang bacain catetannya mundur buat ngitungin turunannya otomatis buat kita.

Bagian 4. NLP, Klasifikasi & Jarak

Embedding: Ubah Makna Jadi Lokasi Peta

Komputer buta sama tulisan “Kucing”. Jadi wajib diubah jadi angka. Tapi nggak boleh asal di-labelin ID (Kucing=1, Anjing=2, Meja=3). Nanti komputernya salah paham ngira nilai Anjing

letaknya persis di tengah antara Kucing dan Meja. Solusinya: Jadikan kata tersebut sebagai koordinat di dalam peta tata surya dimensi tinggi. Kucing dan Anjing letaknya saling nempel deketan, sedangkan Meja letaknya jauh di galaksi lain. Inilah **Embedding**. Karena udah menjelma jadi titik koordinat, “kemiripan” antar kata bisa diukur gampang pakai patokan sudut jarak (*Cosine Similarity*, yang rumus dan logikanya sama persis 100% sama pengukuran *overlap* vektor bra-ket di Fisika Kuantum).

Softmax & Cross Entropy

Buat nebak kelas (klasifikasi), kita butuh output berbentuk probabilitas (persentase total 100%). **Softmax** adalah alat penyaring yang ngubah angka mentah tebakan model jadi format probabilitas mematuhi hukum batas eksponensial. Ini adalah duplikat murni dari *Distribusi Boltzmann* di Fisika Termodinamika. Parameter temperature di LLM itu ya suhu sistem fisik ini; makin di-set panas angkanya, makin acak/liar variasi tebakan modelmu.

Untuk ukur skor errornya kita pakai **Cross-Entropy**. Intinya ini meteran tingkat “Kekagetan”. Kalau model congkak nebak 99% itu kucing padahal di kunci jawaban itu gambar anjing, dia bakal super kaget dan nilai error Loss-nya kena penalti meledak parah. Kalau model bilang 50-50, kagetnya sedang aja.

Bagian 5 & 6. Suara dan Wajah

Suara (Fourier & Konvolusi CNN)

Audio itu grafiknya mulus tiada henti. Biar gampang, kita cacah/iris dan kita preteli frekuensinya pakai **Transformasi Fourier** (memisahkan suara akor gitar ruwet jadi not tangga nada dasar penyusunnya). Setelah diolah jadi gambar visual 2D (Spektrogram), kita deteksi isinya pakai **CNN**. Konsep CNN itu ibarat kita ngebikin sebuah “stempel karet” kecil pendeteksi pinggiran. Lalu stempel ini digeser (disapu pelan-pelan) ke seluruh area gambar. Di mana stempelnya cocok sama polanya, di situ sistemnya menyala (hasil tanggapnya membesar). Teknik berbagi stempel ini menghemat jutaan parameter yang nggak guna.

Wajah (Metric Learning)

Ngebandingin wajah dari warna piksel itu ide bodoh (karena geser lampu senter dikit aja piksel mukamu berubah semua). Cara yang bener: paksa jaringan supaya belajar ngerumusin penggaris baru. “Bikin gambar wajah orang yang sama ngumpul nempel di titik koordinat yang berdekatan, sementara wajah yang beda orang dorong sejauh-jauhnya”. Inilah **Metric Learning**. Modelnya murni sibuk ngekalibrasi penggaris jarak, bukan ngapalin hidung atau mata spesifik. Makanya dia tetep sakti bisa ngenalin muka maling baru walau belum pernah ngelihat mukanya pas di masa *training*.

Bagian 7. LLM & Mesin Pemaham Bahasa

Model LLM nggak memproses satu kalimat utuh, tapi ngebaca **Token** (cacahan patahan kata).

Jantung mesin ini namanya **Attention**. Bayangin kayak lagi buka-buka ensiklopedia di perpustakaan. Setiap kata nulis kata kunci pencarian (*Query*), nempelin judul di punggung punggungnya (*Key*), dan nyodorin isi bukunya kalau ada yang merasa relevan (*Value*). Jadi di dalam satu kalimat, setiap kata sibuk ngelirik kata-kata di sekitarnya dan meracik konteks maknanya. Biar modelnya nggak kebingungan susunan urutan (karena attention ngebaca sekilas semua kalimat secara serentak tanpa peduli urutan awal/akhir), kita harus ngecap/stempel posisi setiap kata pakai alat bantu gelombang sinus-cosinus (lagi-lagi *Deret Fourier*!) yang bertugas jadi ID **Positional Encoding**.

Dan ini rahasia gedanya: LLM kayak ChatGPT itu sepintar itu bukan karena sengaja diprogram buat jadi orang bijak. Dia cuma **mesin autocomplete penebak kata selanjutnya**. Saking ekstrim dan gilanya tekanan pelatihan untuk bisa akurat “menebak kata”, dia tanpa sengaja memaksakan dirinya ikutan belajar nyerep struktur tata bahasa, fakta sejarah, nalar logika, dan gaya penulisan cuma sebagai trik/kunci bantu agar tebakannya selalu jitu.

Bagian 8. JARVIS (Agent System)

Sistem Asisten AI cerdas itu cuma ngulang *looping* diagram kendali jadul: Dengar -> Pahami -> Putuskan (Pilih Tool) -> Eksekusi -> Laporkan -> Ulangi.

Di sistem asisten SYNESIS kita, perintah rumah tangga (matiin lampu, buka file) nggak perlu diterjunin ke LLM raksasa miliaran parameter yang super lelet. Cukup serahkan ke model *classifier* kecil sekelas teri. LLM cuma dibangunin khusus buat ngeladenin tugas nulis/riset yang *open-ended*.

Kalau ngasih instruksi ke LLM supaya dia pakai fitur aplikasi (*Tool Calling*), kadang dia suka sok pinter nulis *syntax* format JSON-nya berantakan/kurang koma. Jangan ngemis mohon-mohon perbaikan lewat Prompting! Gunakan **Grammar Constraint**: kita gembok paksa batasan rel keretanya. Secara arsitektur, model fisik murni diblokir dari ngetik token yang merusak format. Hasilnya? JSON valid 100%.

Satu lagi: **Selalu bikin fitur keamanan duluan!** Kalau bikin bot yang bisa baca tulis hapus file di sistem, bikin dulu lapis perlindungan (tombol abort, blacklist folder, konfirmasi admin) sebelum nyemplung bikin tool serangnya. Jangan sampai nangis garuk-garuk aspal pas eksperimen bodohmu secara ajaib menghapus skripsi atau file OS gara-gara salah *prompt*.

Bagian 9. Kamus Fisika ke Machine Learning

Buat anak Eksakta/Fisika, ini jalan tol curangmu karena nama-nama panggung mentereng di ML aslinya cuma konsep jadul Fisika yang diganti casingnya:

Istilah Fisika / Matdas Kamu	Nama Panggunanya di ML
Energi Potensial Terendah	Loss Minimum
Kelereng turun bukit	Gradient Descent

Istilah Fisika / Matdas Kamu	Nama Panggunnya di ML
Potensial Harmonik & Osilasi Pegas	Permukaan Loss MSE & Limit Learning Rate
Hukum Hooke (Gaya Pemulih Pegas)	Regularisasi L2
Aturan Rantai Kalkulus (Dari Belakang)	Backpropagation
Overlap Keadaan Bra-Ket Kuantum	Cosine Similarity / Dot Product
Distribusi & Konstanta Boltzmann	Softmax (beserta parameter Temperature)
Penurunan Entropi Termo Shannon	Cross-Entropy Loss
Fungsi Deret Gelombang Fourier	Positional Encoding
Limit Frekuensi Teorema Nyquist	Sample Rate Audio (anti Aliasing)
Lingkar Kendali Umpan Balik (Kontrol)	Agent System Loop

Kalau stuck atau pusing, tinggal liat tabel ini dan mikir, “*Oh, ini mah cuma sekedar nyari amplitudo gelombang aja tapi namanya di-inggris-in*”.

Bagian 10. Tes Kejujuran (Apakah Lu Beneran Paham?)

Dunia AI / Deep Learning itu gudangnya orang-orang *fomo* (ikut-ikutan) pemuja kosakata elit (*Cargo Cult*). Ngomongin Backprop, Attention, atau Tensor tiap hari, tapi pas dikasih *error bug* dikit langsung bengong. Biar kamu nggak jadi kaum halu penipu diri sendiri, saring pakai **Feynman Technique** ini tiap beres belajar:

1. **Jelasin ulang tanpa istilah gaib.** Coba terangin apa itu ‘Gradient Descent’ ke nenekmu atau bocah SMP tanpa satupun nyebut kata “Loss”, “Iterasi”, atau “Optimasi”. Kalau lu terbata-bata nyari kosakatanya, berarti lu cuma hafal stempelnya, bukan paham cara kerjanya.
2. **Tabrak dari sudut lain.** Kalau kamu ngerti rumusnya, jawab spontan: Kenapa turunan di gradien kok *wajib* dikasih tanda minus? Kenapa LR nggak dilebur jadi satu aja sama rumus aslinya? Kalau pertanyaan digeser dikit kamu goyah, fix pemahamanmu sebatas urutan hafalan tutorial.
3. **Rusak sengaja sistemnya.** Ubah nilai temperature LLM jadi absolut 0. Naikin angka Learning Rate jadi 1000 kali lipat. Hapus semua fungsi aktivasi ReLU. Ramalkan/tebak dalem hati *chaos* aneh macam apa yang bakal terjadi **sebelum** ngeklik run. Kalau tebakan lu akurat, selamat, insting teknis (intuisi mekanik) lu beneran udah jadi.